ITC INFORMATION AND COMMUNICATIONS TECHNOLOGY ACADEMY

MODULO: X.Y
UNITÀ: X

Dipartimento di Informatica
Sapienza Università di Roma

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Virtualization & containers

Docker

Virtual Machine

Virtual machines (VMs) are created through a process called virtualization.

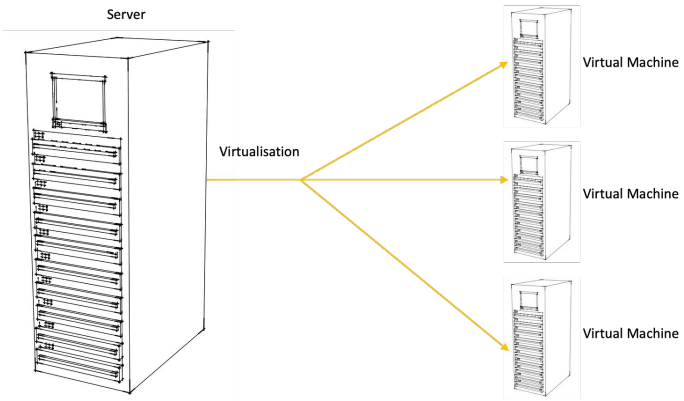
Virtualization is a technology that allows you to create multiple simulated environments (e.g., a server, storage, or a network) on a single physical machine.

These virtual environments behave as if they are independent, separate entities, even though they share the resources of the underlying physical system.

Virtual Machine (2)

VMs virtualize the hardware. This simply means that a VM takes a single piece of hardware – a server – and creates virtual versions of other servers running their own operating systems. Physically, it is just a single piece of hardware.

Logically, multiple virtual machines can run on top of a single piece of hardware. This is essentially one or more computers running within a computer, as shown below.



Container

A container is a lightweight, standalone, and executable software package that includes everything needed to run a piece of software, including the code, runtime, system tools, and libraries.

Containers are designed to isolate applications and their dependencies, ensuring that they can run consistently across different environments.

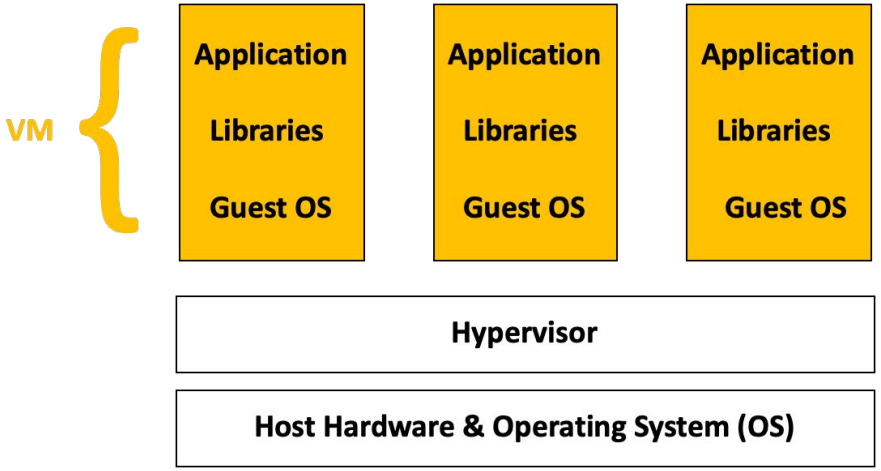
Whether the application is running from your computer or in the cloud, the application behaviour remains the same.

VM vs Container

VM virtualization:

- VMs
- Hypervisor
- Host hardware and OS (i.e., the physical machine used to create virtual machines).

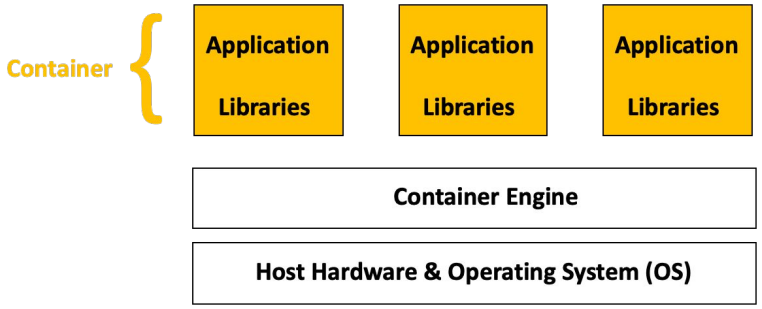
Each VM has its operating systems (OS), libraries and applications running in a single physical machine



VM vs Container (2)

Containers:

- Each container uses a single OS to create a virtual application and its libraries.
- Container engine that virtualizes the operating system.
- Shared hardware and OS provided by the host system.



VM vs Container (3)

VM

- **Portability**: VMs are typically less portable due to differences in underlying OS environments. Moving VMs between different hypervisors or cloud providers can be more complex.

- **Efficiency**: VMs have a higher resource overhead due to the need to run a full OS instance for each VM. This can lead to larger memory and storage consumption. This in turn can have a negative effect on performance and startup times of the virtual machine

Container

- **Portability**: Containers are designed to be platform-independent. They can run on any system that supports the container runtime regardless of the underlying operating system. This makes it easier to move applications between different environments.

- **Efficiency**: Containers share the host system's operating system, which reduces the overhead of running a virtual machine with multiple operating systems. This leads to more efficient resource utilization and allows for a higher density of applications that can run on a single host.

Container - additional benefits

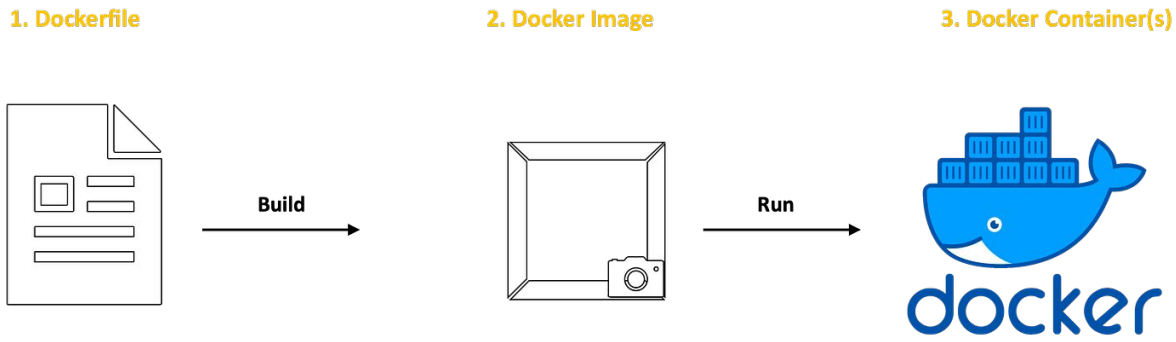
- **Consistency**: Containers package all the necessary components, including the application code, runtime, libraries, and dependencies, into a single unit. This eliminates the "it works on my machine" problem and ensures that the application runs consistently across different environments, from development to production.

- **Isolation**: Containers provide a lightweight and isolated environment for running applications. Each container encapsulates the application and its dependencies, ensuring that they do not interfere with each other. This isolation helps prevent conflicts and ensures consistent behaviour across different environments.

- **Fast Deployment:** Containers can be created and started quickly, often in a matter of seconds. This rapid deployment speed is particularly beneficial for applications that need to rapidly scale up or down based on demand

Docker

- Docker is simply a tool for creating and managing containers.
- Docker has two main concepts that are useful to understand:
  - **Dockerfile:** set of instructions for building a Docker Image.
  - **Docker Image:** template for creating Docker containers. It contains all the necessary code, runtime, system tools, libraries, and settings required to run a software application.



Exercise 1

To test your installation, try to run your first container.

1. Open your terminal into your Linux environment.
2. Run the following command:

```
$ docker run -it hello-world:latest
```

What is happened?

Registry & Docker Hub

A Docker Registry is a tool for storing and distributing Docker images.

The Docker community offers a public platform to share and provide docker images (see https://hub.docker.com/).

When you try to use a docker image, your docker environment tries to find out if the requested image can be found locally. Otherwise, it tries to pull the requested image from Docker Hub.

To see all images on your local machine:

```
$ docker images
```

Docker basic commands

See all running containers:

```
$ docker ps
```

See all containers (up end exited):

```
$ docker ps -a
```

Run a new container (remaining up)

```
$ docker run -it <image-name:image-tag> bash
```

Docker basic commands (2)

Stop a container:

```
$ docker stop <container-name>
```

Remove a container:

```
$ docker rm <container-name>
```

Remove a Docker image

```
$ docker rmi <image-name:image-tag>
```

Exercise 2

New, create your Python environment and your Python app!

Step 1. Create the following files:

- Create Dockerfile
- Create python_requirements.txt
- Create app.py

Exercise 2 (2)

Step 2. Write the Dockerfile content:

Dockerfile > FROM

```
FROM python:3.11.11

COPY python_requirements.txt /
RUN pip install -r python_requirements.txt

RUN mkdir /app
COPY app.py /app
WORKDIR /app
```

Exercise 2 (3)

Step 3. Write the python_requirements.txt content:

```
numpy==2.2.4
```

Exercise 2 (4)

Step 4. Write the app.py content:

```python
import numpy as np



def main():
    matrix = np.array([
        [1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]
    ])

    print("Matrice di partenza: ")
    print(matrix)

    print("\n\nMatrice trasposta:")
    transposed_matrix = matrix.T
    print(transposed_matrix)


if __name__ == '__main__':
    main()
```

Exercise 2 (5)

Step 4 – Build the docker image

Run the following command:

```
$ docker build . -t dockerized-pythonapp:latest
```

Step 5 – Run the container

Run the following command:

```
$ docker run -it dockerized-pythonapp:latest
python app.py
```

Docker basic commands (3)

Execute a command running a new container:

```
$ docker run -it <image-name:image-tag> <command>
```

Execute a command using a running container

```
$ docker exec -it <container-name> <command>
```

Connect to the container filesystem

```
$ docker exec -it <container-name> bash
```

Exercise 3

1. Create a simple project structure.
2. Create a python_requirments.txt to install python packages.
3. Create app.py to develop a Python app that uses the installed package.
4. Create a Dockerfile based on the previous example to build your docker image.
5. Run the container to execute your Python app.

Multiple services

Docker is often used to deploy complex applications or environments with more than a single container (i.e., multiple services, one per container).

However, managing each container (one at hand) without an automated orchestration procedure may not be practicable.

In addition, multiple containers typically communicate through dedicated virtualized networks, e.g., a web application with the application and the database, which requires manually creating the network and configuring it properly.

Docker compose

Docker Compose is a tool for defining and running multi-container applications.

The docker compose simplifies the control of your entire application, making it easy to manage services, networks, and volumes through a single YAML configuration file.

With a single command, you create and start all the services from your configuration file.

See ITS ICT Docker ecosystem

Now, we explore the ITS ICT Docker ecosystem.

https://github.com/ITS-ICT-Academy/sw_development